

CS 188: Inteligência Artificial

Problemas de Satisfação de Restrições



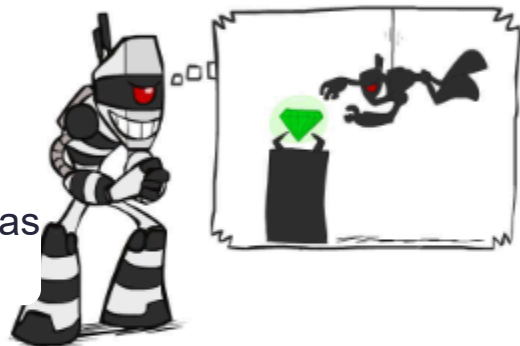
Instrutores: Peyrin Kao e Emma Pierson

Universidade da Califórnia, Berkeley

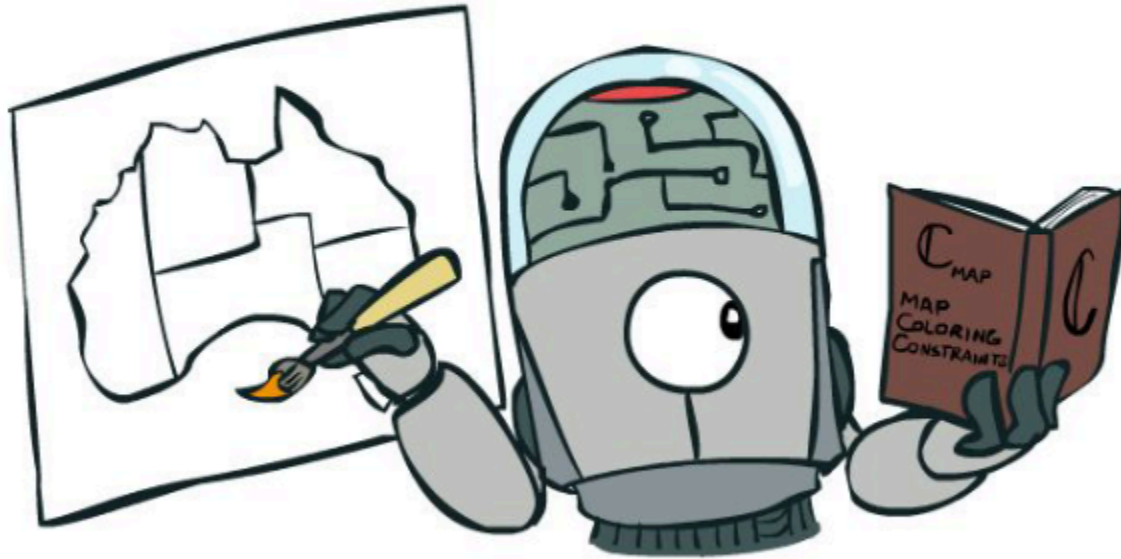
[Esses slides foram criados por Dan Klein e Pieter Abbeel para o CS188 Introdução à IA na UC Berkeley (ai.berkeley.edu).]

O que é Busca?

- Suposições sobre o mundo: um único agente, ações determinísticas, totalmente observadas estado, espaço discreto de estados
- Planejamento: sequências de ações
 - O caminho para o objetivo é o mais importante. Caminhos têm vários custos, profundidades. ▪ Heurísticas fornecem orientações específicas para cada problema.
- Identificação: atribuições a variáveis
 - O objetivo em si é importante, não o caminho. ▪ Todos os caminhos na mesma profundidade (para algumas formulações) ▪ CSPs são uma classe especializada de problemas de identificação

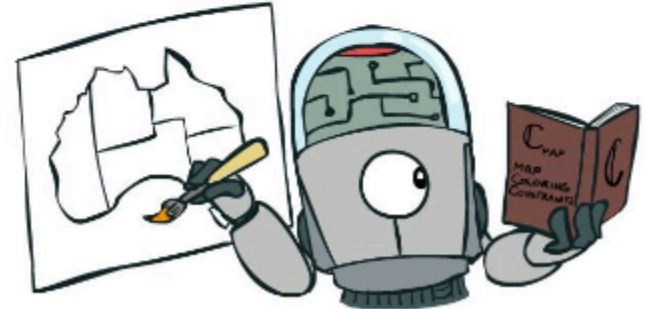
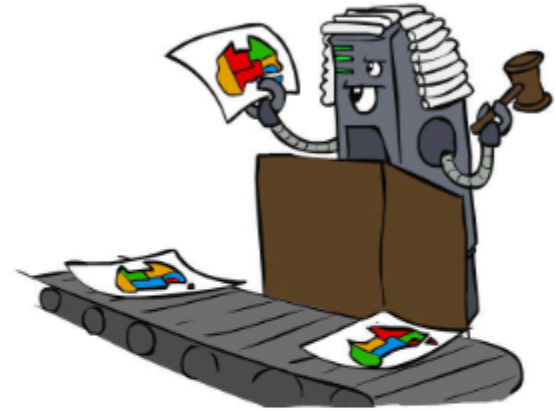


Problemas de Satisfação de Restrições

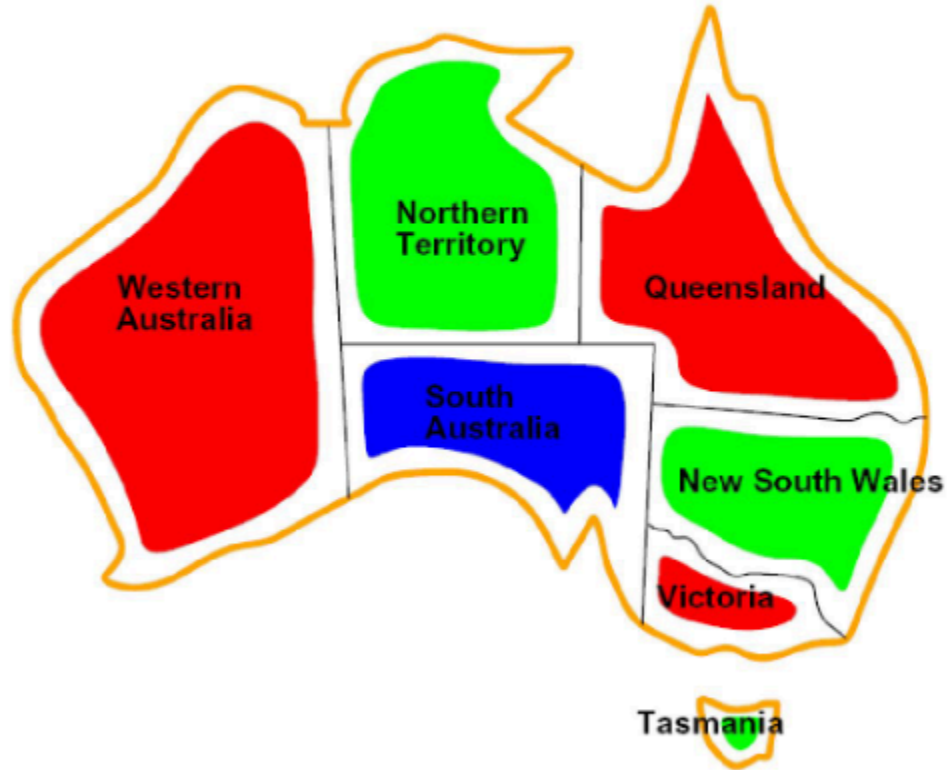


Problemas de Satisfação de Restrições

- Problemas padrão de busca: ▪ Estado é uma "caixa-preta": estrutura ▪ de dados arbitrária Teste de objetivo pode ser qualquer função sobre estados ▪ Função sucessora também pode ser qualquer coisa ▪
- Problemas de satisfação de restrições (CSPs): ▪ Um subconjunto especial de problemas ▪ de busca Estado é definido por variáveis X_i com valores de um domínio D (às vezes D depende de i) ▪ Teste de objetivo é um conjunto de restrições que especificam combinações permitidas de valores para subconjuntos de variáveis ▪
- Exemplo simples de um formal Linguagem ▪ de representação Permite algoritmos de uso geral úteis com mais potência do que algoritmos de busca padrão



Exemplos de CSP



Exemplo: Coloração de Mapas

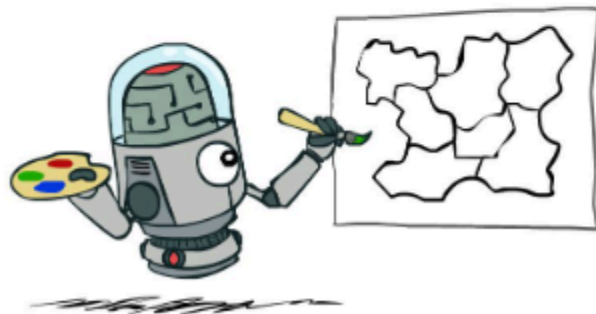
- Variáveis: ▪ Domínios: ▪ Restrições: regiões adjacentes devem ter cores diferentes

Implícito: $WA \neq NT$

Explícito: $(WA, NT) \in \{(red, green), (red, blue), \dots\}$

- Soluções são atribuições que satisfazem todas as restrições, por exemplo:

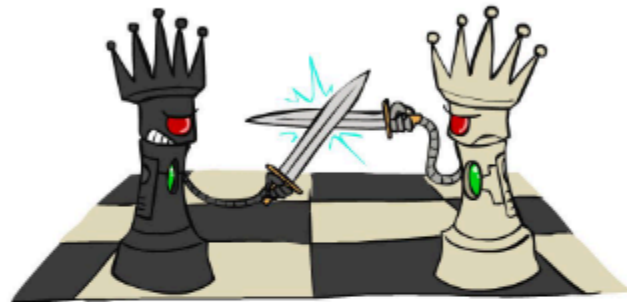
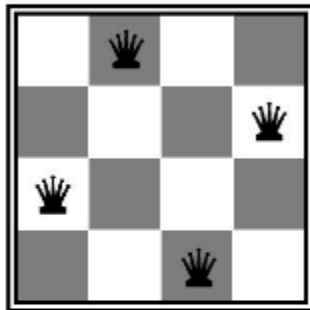
$\{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green\}$



Exemplo: N-Damas

- Formulação 1:

- Variáveis: X_{ij}
- Domínios: $\{0, 1\}$
- Restrições



$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{i+k, j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{i+k, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\sum_{i,j} X_{ij} = N$$

Exemplo: N-Damas

- Formulação 2:

- Variáveis: Q_k

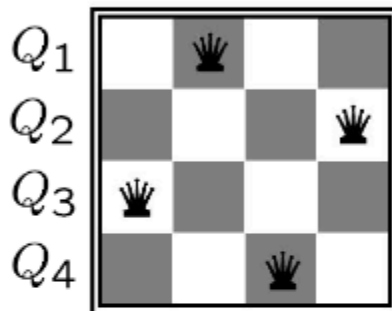
Domínios: $\{1, 2, 3, \dots, N\}$

Restrições:

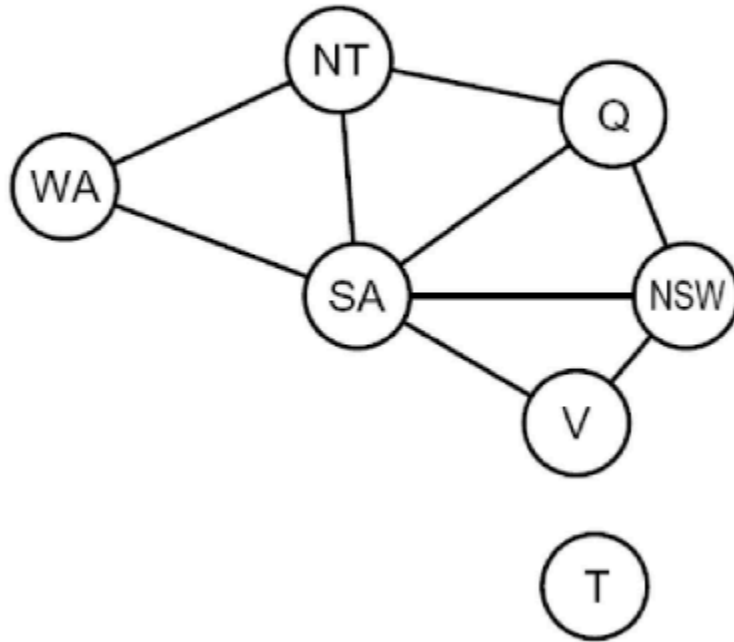
Implícito: $\forall i, j \text{ non-threatening}(Q_i, Q_j)$

Explícito: $(Q_1, Q_2) \in \{(1, 3), (1, 4), \dots\}$

...

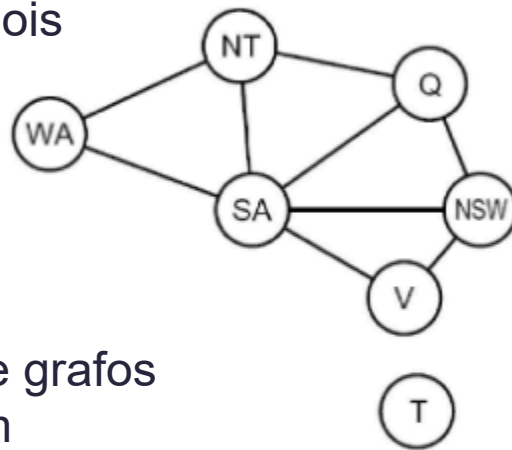


Grafos de restrições



Grafos de restrições

- CSP binário: cada restrição relaciona (no máximo) dois Variáveis
- Grafo binário de restrições: nós são variáveis, arcos Restrições de mostrar
- Algoritmos de CSP de uso geral usam a estrutura de grafos para acelerar a busca. Por exemplo, a Tasmânia é um Subproblema independente!



Exemplo: Criptoaritmética

- Variáveis:

$F T U W R O X_1 X_2 X_3$

- Domínios:

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

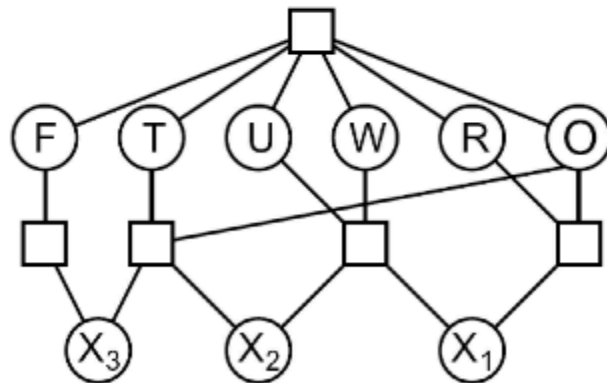
- Restrições:

$\text{alldiff}(F, T, U, W, R, O)$

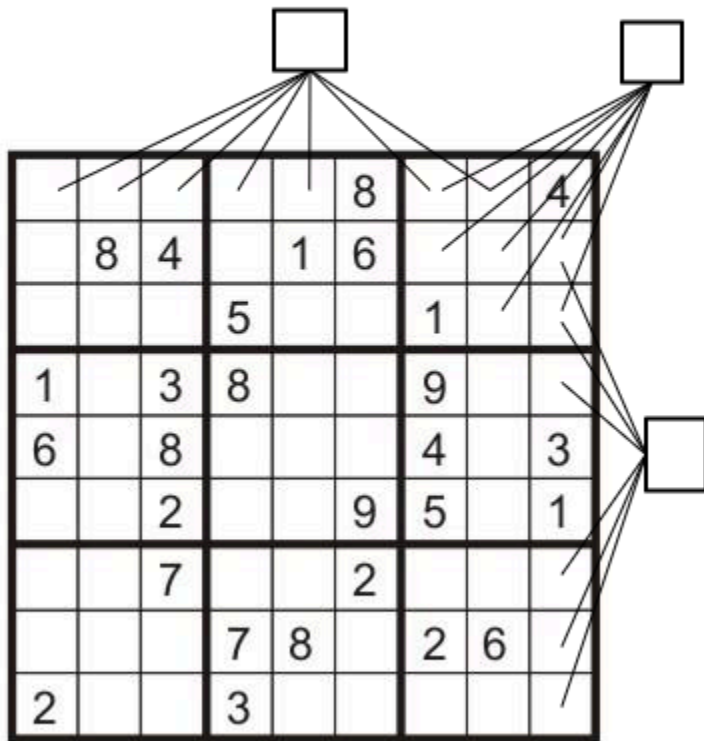
$O + O = R + 10 \cdot X_1$

...

$$\begin{array}{r} T W O \\ + T W O \\ \hline F O U R \end{array}$$



Exemplo: Sudoku



- Variáveis:
- Cada quadrado (aberto)
- Domínios: $\{1, 2, \dots, 9\}$

Restrições:

Alldiff de 9 vias para cada coluna

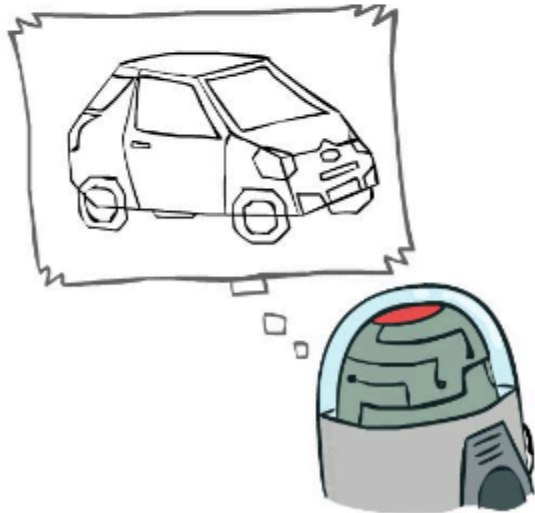
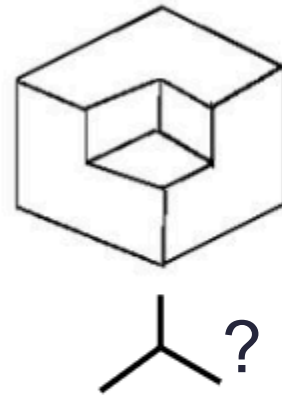
Alldiff de 9 vias para cada fileira

Alldiff de 9 vias para cada região
(ou pode ter várias restrições de desigualdade por pares)

Alguns quadrados já
preenchidos (restrições unárias)

Exemplo: O Algoritmo da Valsa

- O algoritmo Waltz serve para interpretar desenhos lineares de poliedros sólidos como objetos 3D. Um exemplo inicial de computação de IA apresentado como um CSP



- Abordagem: ▪ Cada interseção é uma variável ▪ Interseções adjacentes impõem restrições umas às outras ▪ Soluções são fisicamente realizáveis 3 Interpretações

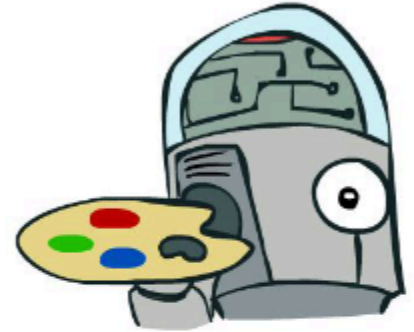
Variedades de CSPs e Restrições



Variedades de CSPs

- Variáveis ▪ Discretas Domínios ▪ finitos
Tamanho d significa $O(d^n)$ atribuições ▪
completas Por exemplo, CSPs booleanos,
incluindo satisfatibilidade booleana

- (NP-completo) ▪ Domínios infinitos (inteiros, cadeias, etc.) ▪
Por exemplo, escalonamento de trabalhos, variáveis são
horários de início/fim para cada tarefa ▪ Restrições lineares
solúveis, não lineares indecidíveis



- Variáveis ▪ contínuas Por exemplo, horários de início/fim
para observações ▪ do Telescópio Hubble Restrições
lineares solucionáveis em tempo polinomial por métodos LP
(veja CS170 para um pouco dessa teoria)



Variedades de restrições

- Variedades de Restrições ▪ Restrições unárias envolvem uma única variável (equivalente a domínios redutores), por exemplo:

$SA \neq \text{green}$

- Restrições binárias envolvem pares de variáveis, por exemplo:

$SA \neq WA$

- Restrições de ordem superior envolvem 3 ou mais variáveis:
por exemplo, restrições criptoaritméticas de colunas

- Preferências (restrições suaves): ▪ Por exemplo, vermelho é melhor que verde ▪ Frequentemente representável por um custo para cada atribuição ▪ de variável Dá problemas ▪ de otimização restrita (Vamos ignorar isso até chegarmos às redes de Bayes)



CSPs do Mundo Real

▪ Problemas de agendamento: por exemplo, quando podemos nos reunir? ▪ Problemas de horário: por exemplo, qual disciplina é oferecida quando e onde? ▪ Problemas de trabalho: por exemplo, quem ensina qual turma, quem senta em qual mesa ▪ Medicina: radioterapia, designação de turnos de enfermeira + médico, agendamento de operações salas, escolha de cronogramas ▪ de tratamento de drogas Configuração ▪ de hardware Agendamento ▪ de transporte Programação ▪ de fábrica Disposição ▪ do circuito Diagnóstico ▪ de falhas ... Muito mais!

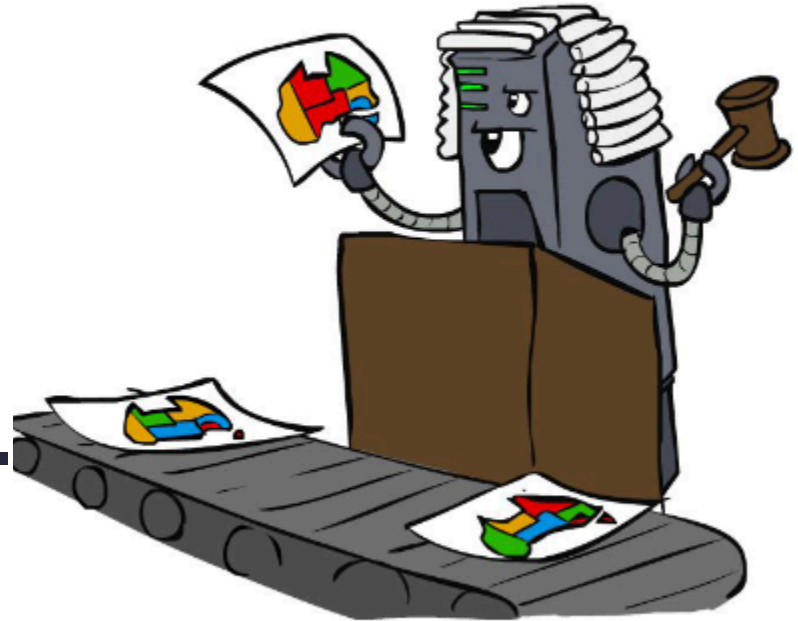
▪ Muitos problemas do mundo real envolvem variáveis de valor real...

Resolvendo CSPs



Formulação Padrão de Busca

- Formulação padrão de busca dos CSPs
- Estados definidos pelos valores atribuídos até agora (atribuições parciais)
- Estado inicial: a atribuição vazia, $\{\}$
- Função sucessora: atribuir um valor a uma variável
- não atribuída
- Teste objetivo: a atribuição atual é completa e satisfaz todas as restrições
- Vamos começar com a abordagem direta e ingênua, depois melhorá-la

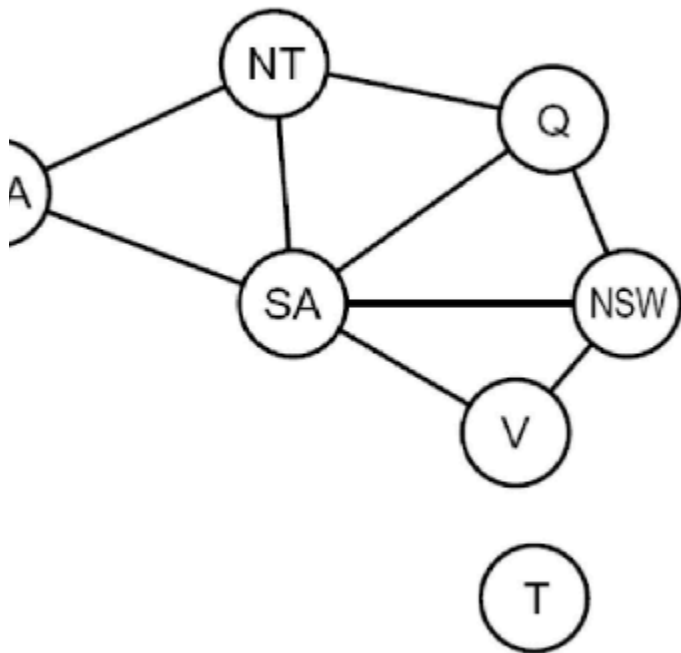


Métodos de Busca

- O que o BFS faria? ▪ Raiz: Atribuição vazia: {} ▪ Varrer sobre atribuições parciais para 1 variável: {WA = R}, {WA = B}, {WA = G}, {NT = R}... ▪ Depois, atribuições parciais a 2 variáveis: {WA = R, NT = R}, {WA = R, NT = B}... ▪ ... Tem que passar por toda a árvore! ▪ Todas as soluções estão no final = ruim! ▪ O que o DFS faria? ▪ Talvez mais sensato? Pelo menos começa verificando

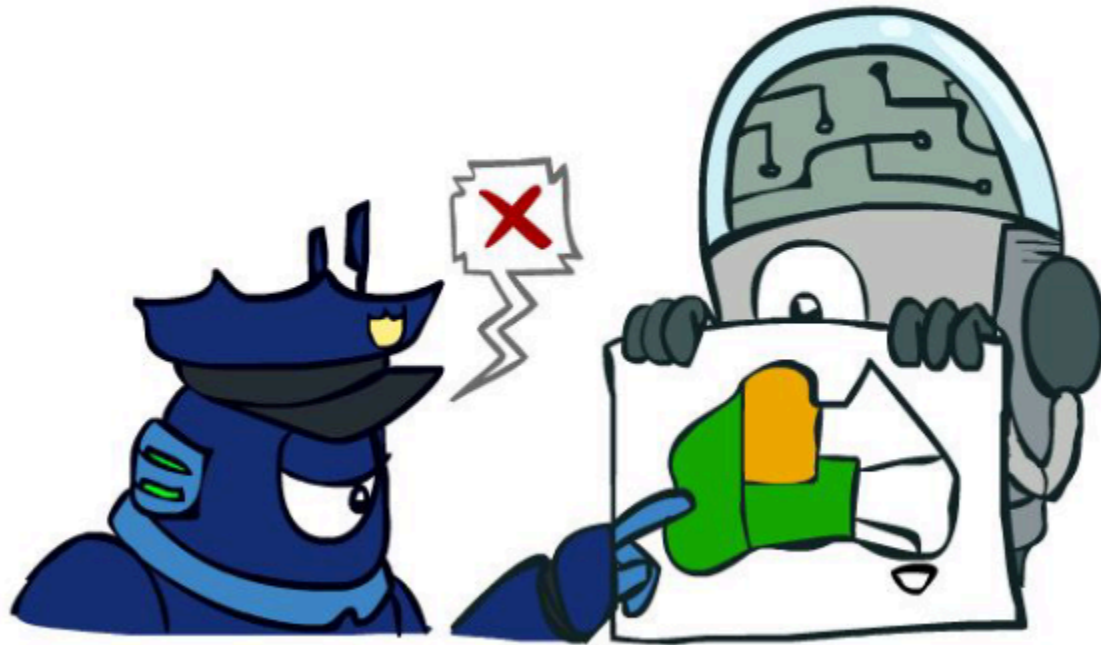
soluções completas▪.

Vamos dar uma olhada...



Demo: DFS

Busca de retrocesso

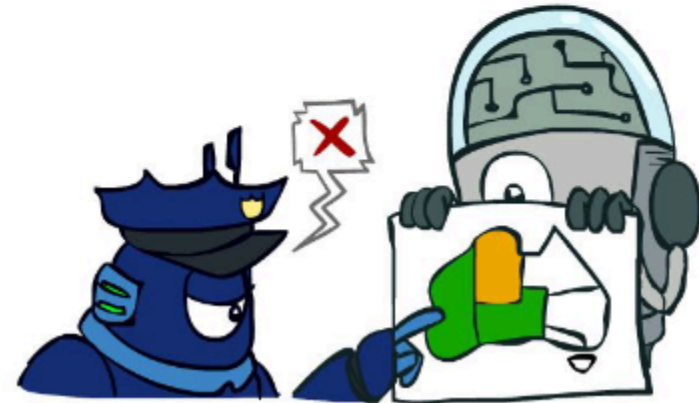
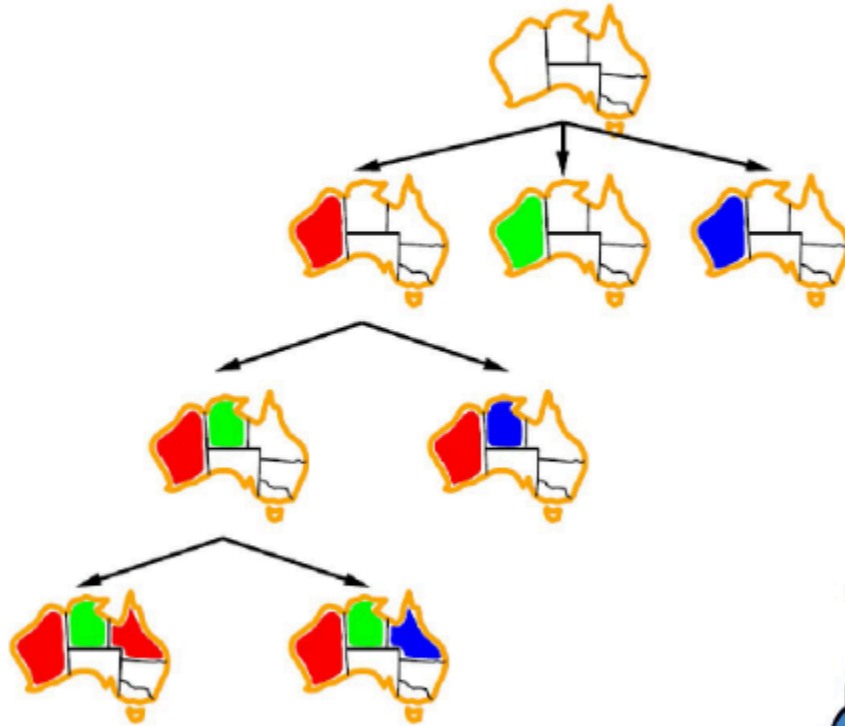


Busca de retrocesso

▪ A busca por retrocesso é o algoritmo básico e desinformado para resolver CSPs ▪ Ideia 1: Uma variável por vez ▪ As atribuições de variáveis são comutativas, então fixar a ordenação, ou seja, [WA = vermelho depois NT = verde = verde] igual a [NT = verde depois WA = vermelho] ▪ Basta considerar as atribuições a uma única variável em cada etapa ▪ Ideia 2: Verifique as restrições conforme avança, ou seja, considere apenas valores que não conflitam com atribuições anteriores Pode ser necessário fazer algum cálculo para verificar as restrições ▪ "Teste de objetivo incremental" ▪ Busca em profundidade com essas duas melhorias  é chamado de busca por retrocesso (não é o melhor nome) ▪ Pode resolver n-queens por $n \approx 25$



Exemplo de retrocesso



Busca de retrocesso

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

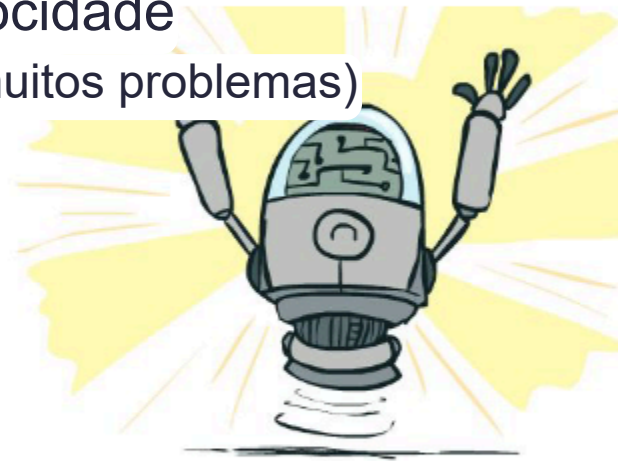
function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

- Retrocesso = DFS + ordenação de variáveis + falha na violação
- Quais são os pontos de escolha? Ordem das variáveis, ordem de Valores.

Demo: retrocesso

Melhorando o Backtracking

- Ideias de uso geral dão grandes ganhos de velocidade
 - (Ao contrário das heurísticas A^* , elas se aplicam a muitos problemas)
- Ordenação:
 - Qual variável deve ser atribuída em seguida? ▪
Em que ordem seus valores devem ser testados?
- Filtragem: Podemos detectar falhas inevitáveis precocemente?
- Estrutura: Podemos explorar a estrutura do problema?

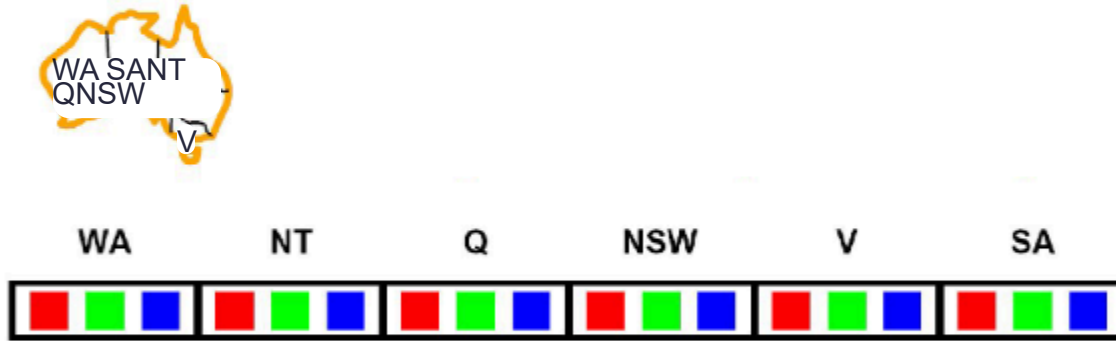


Filtragem



Filtragem: Verificação Direta

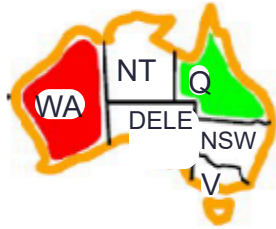
- Filtragem: Acompanhe domínios para variáveis não atribuídas e risce opções ▪ ruins
- Verificação direta: Risce valores que violem uma restrição quando adicionados ao existente designação



Demo: Verificação Direta

Filtragem: Propagação de Restrições

- A verificação direta propaga informações de variáveis atribuídas para não atribuídas, mas não fornece detecção precoce para todas as falhas:

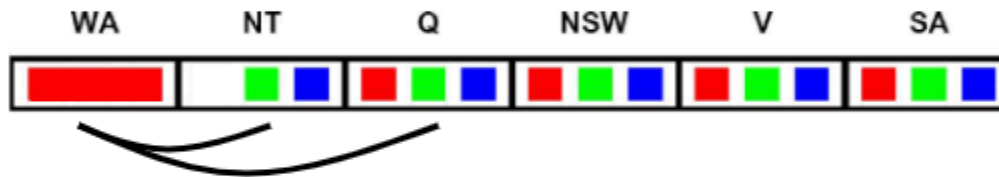
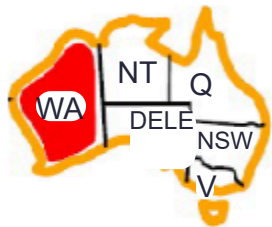


WA	NT	Q	NSW	V	SA

- NT e SA não podem ser ambos azuis! ▪ Por que ainda não detectamos isso? ▪ Propagação de restrições: razão de restrição para restrição

Consistência de um único arco

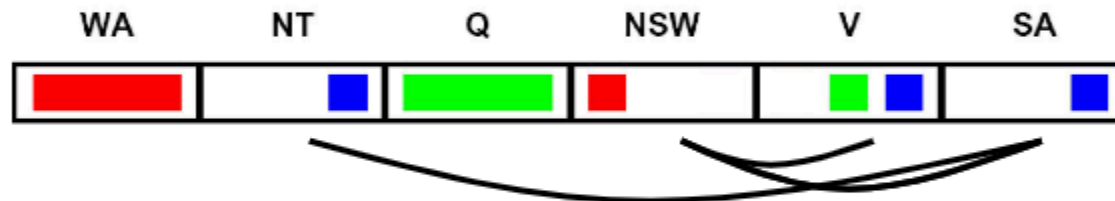
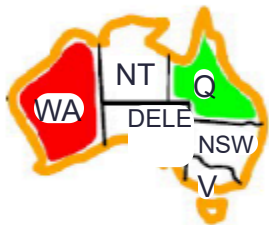
- Um arco $X \rightarrow Y$ é consistente se para cada x na cauda existe algum y na cabeça que poderia ser atribuído sem violar uma restrição (cabeça = a seta!)



- Verificação direta: Impor consistência dos arcos apontando para cada nova atribuição. Mas outros tipos de algoritmos de consistência de arco são possíveis!

Consistência de arco de um CSP inteiro

- Uma forma simples de propagação garante que todos os arcos sejam consistentes:



- Importante: Se X perder um valor, vizinhos de X precisam ser verificados novamente!
- Consistência de arco detecta falha antes da verificação direta.
- Pode ser executado como pré-processador ou após cada atribuição.
- Um grafo é consistente com arco se todos os arcos do grafo forem consistentes.
- Qual é a desvantagem de impor consistência de arco?

Lembre-se:
Excluir de
A cauda!

Imposição da Consistência de Arco em um CSP

```
function AC-3(csp) returns the CSP, possibly with reduced domains
  inputs: csp, a binary CSP with variables  $\{X_1, X_2, \dots, X_n\}$ 
  local variables: queue, a queue of arcs, initially all the arcs in csp

  while queue is not empty do
     $(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$ 
    if REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) then
      for each  $X_k$  in NEIGHBORS[ $X_i$ ] do
        add  $(X_k, X_i)$  to queue



---


function REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) returns true iff succeeds
  removed  $\leftarrow$  false
  for each  $x$  in DOMAIN[ $X_i$ ] do
    if no value  $y$  in DOMAIN[ $X_j$ ] allows  $(x, y)$  to satisfy the constraint  $X_i \leftrightarrow X_j$ 
      then delete  $x$  from DOMAIN[ $X_i$ ]; removed  $\leftarrow$  true
  return removed
```

- Tempo de execução: $O(n^2d^3)$, pode ser reduzido a $O(n^2d^2)$ ▪ ... mas detectar todos os possíveis problemas futuros é NP-difícil – por quê? Porque isso nos permitiria resolver problemas NP-difíceis (como satisfatibilidade).

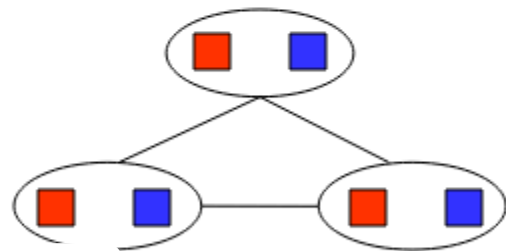
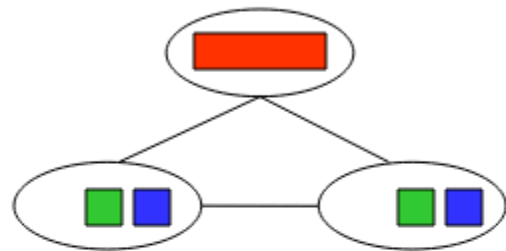
Limitações da Consistência do Arco

- Após impor o arco

consistência:

- Pode sobrar uma solução
- Pode ter várias soluções restantes
- Não pode sobrar nenhuma solução (e não saber)

- A consistência de arco ainda é válida
Dentro de uma busca de retrocesso!



O que deu errado aqui?

[Demo: coloração -- checagem frontal]

[Demo: coloração -- consistência do arco]

Demo: Retrocesso com verificação direta

Demo: Retrocesso com consistência de arco

Ordenação

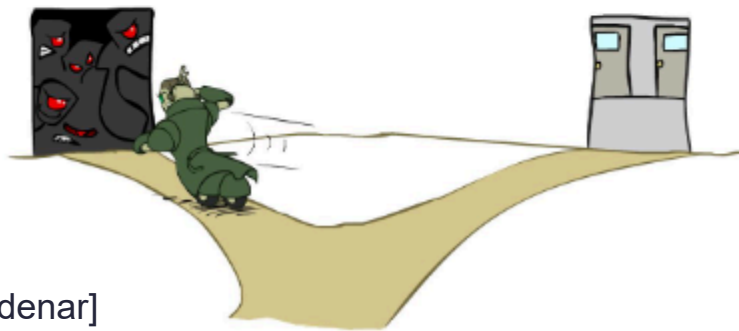


Orden: Valores Mínimos Restantes

- Ordenação das variáveis: Valores mínimos restantes (MRV):
 - Escolha a variável com menos valores legais restantes em seu domínio



- Por que mínimo em vez de máximo?
- Também chamado de ordenação "variável mais restrita" "▪ Fail-fast"



[Demo: colorir – voltar + AC + ordenar]

Demo: Backtracking + Pedido de AC + MRV

Ordenação: Valor Mínimo Restritivo

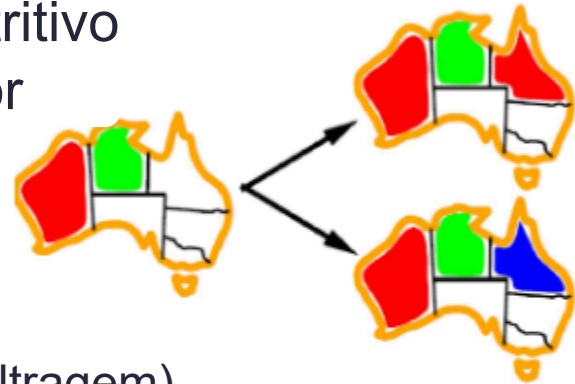
- Ordenação de Valores: Valor Mínimo Restritivo

Dada a escolha de variável, escolha a menor

valor restritivo, ou seja, aquele que exclui o menor número de valores em

as variáveis restantes Note que pode levar algum cálculo para

Determina isso! (Por exemplo, reexecutando filtragem)



- Por que menos e não a maioria? ▪

Combinar essas ideias de ordenação torna

1000 rainhas possíveis

